Monitoring & Observability

Prometheus, Grafana & Alerting for KVM Infrastructure

Replace vCenter monitoring with open-source observability. Prometheus + Grafana + Alertmanager provides deeper metrics, better dashboards, and flexible alerting — all at zero license cost. This guide covers exporter setup, dashboard templates, alerting rules, and vCenter-to-Grafana migration.

Prometheus | Grafana | node_exporter | libvirt_exporter | Alertmanager | Loki | $0 License

Monitoring Stack Architecture

vCenter replacement with Prometheus + Grafana + Loki — open-source, production-proven.

VMware vCenter vs Open-Source Monitoring

Feature	vCenter / vROps	Prometheus + Grafana	Cost (vCenter)	Cost (Prometheus)	Winner
Host metrics	**Built-in**	**node_exporter**	Included in vSphere	$0	**Tie**
VM metrics	**Built-in**	**libvirt_exporter**	Included	$0	**Tie**
Custom dashboards	Limited	**Unlimited (Grafana)**	vROps: $70K+	$0	**Grafana**
Alerting	vROps alerts	**Alertmanager (Slack, PagerDuty, email)**	vROps required	$0	**Alertmanager**
Log aggregation	vRealize Log Insight	**Loki + Promtail**	$50K+	$0	**Loki**
Retention	30 days (default)	**Years (configurable)**	Storage adds cost	Local disk	**Prometheus**
API/Query	SOAP API (complex)	**PromQL (powerful)**	—	—	**PromQL**
Annual Cost			**$120K+ (vROps+Log)**	**$0**	

Stack Components

```
# Install monitoring stack on dedicated host (or use containers)
# 1. Prometheus (metrics collection + storage)
dnf install prometheus2 && systemctl enable --now prometheus

# 2. Grafana (dashboards + visualization)
dnf install grafana && systemctl enable --now grafana-server

# 3. node_exporter (host metrics — install on EVERY KVM host)
dnf install node_exporter && systemctl enable --now node_exporter

# 4. libvirt_exporter (VM metrics — install on EVERY KVM host)
# github.com/Tinkoff/libvirt-exporter
./libvirt_exporter --libvirt.uri="qemu:///system" &

# 5. Alertmanager (alerting — Slack, PagerDuty, email)
dnf install alertmanager && systemctl enable --now alertmanager
```

```
# 6. Loki + Promtail (log aggregation — optional)
# Install via Docker or Helm for simplicity
```

Prometheus Configuration

Scrape configuration for KVM hosts and VM metrics.

prometheus.yml

Scrape Configuration

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'kvm-hosts'
    static_configs:
      - targets:
          - 'kvm-host-01:9100'
          - 'kvm-host-02:9100'
          - 'kvm-host-03:9100'
        labels:
          cluster: production

  - job_name: 'libvirt'
    static_configs:
      - targets:
          - 'kvm-host-01:9177'
          - 'kvm-host-02:9177'
          - 'kvm-host-03:9177'

  - job_name: 'ceph'
    static_configs:
      - targets:
          - 'ceph-mon-01:9283'

rule_files:
  - 'rules/kvm-alerts.yml'

alerting:
  alertmanagers:
    - static_configs:
        - targets: ['localhost:9093']
```

Key Metrics Available

Exporter	Metric	What It Measures
node	`node_cpu_seconds_total`	Host CPU usage per core
node	`node_memory_MemAvailable_bytes`	Available host memory
node	`node_disk_io_time_seconds_total`	Disk I/O utilization
node	`node_network_receive_bytes_total`	Network traffic
libvirt	`libvirt_domain_info_cpu_time_seconds`	Per-VM CPU usage
libvirt	`libvirt_domain_info_memory_usage_bytes`	Per-VM memory usage
libvirt	`libvirt_domain_block_stats_read_bytes`	Per-VM disk read
libvirt	`libvirt_domain_interface_stats_receive_bytes`	Per-VM network rx

Grafana Dashboards

Pre-built dashboard templates for KVM host and VM monitoring.

Dashboard Templates

KVM Host Overview Dashboard

- **Row 1:** Host CPU usage (per core), load average, uptime
- **Row 2:** Memory usage (used/cached/free), swap usage
- **Row 3:** Disk I/O (IOPS, throughput, latency), space usage
- **Row 4:** Network traffic (rx/tx per interface), errors
- **Row 5:** VM count, total vCPUs, total VM memory
- **Variables:** host selector, time range
- Grafana ID: 1860 (Node Exporter Full)

VM Performance Dashboard

- **Row 1:** VM list table (name, state, CPU, memory, disk)
- **Row 2:** Selected VM CPU usage over time
- **Row 3:** Selected VM memory usage + balloon status
- **Row 4:** Disk IOPS + throughput + latency per disk
- **Row 5:** Network rx/tx per interface
- **Variables:** host selector, VM name selector
- Custom dashboard using libvirt_exporter metrics

Capacity Planning Dashboard

- **Gauge:** Overall cluster CPU utilization %
- **Gauge:** Overall cluster memory utilization %
- **Gauge:** Storage pool utilization %
- **Trend:** 30-day CPU/memory/storage growth rate
- **Forecast:** Days until 80% capacity (predict_linear)
- **Table:** Top 10 resource-consuming VMs

PromQL Examples

```
# Host CPU usage percentage
100 - (avg by (instance) (
  rate(node_cpu_seconds_total{mode="idle"}[5m])
) * 100)

# VM CPU usage percentage
rate(libvirt_domain_info_cpu_time_seconds[5m])
  / on(domain) libvirt_domain_info_vcpus * 100

# Disk space forecast (days until full)
predict_linear(
  node_filesystem_avail_bytes{
    mountpoint="/var/lib/libvirt"
  }[7d], 86400 * 30
) / 1e9

# Memory pressure
1 - (node_memory_MemAvailable_bytes
  / node_memory_MemTotal_bytes)
```

Alerting Rules

Production-ready alerting rules for KVM infrastructure.

Critical Alert Rules (kvm-alerts.yml)

```
groups:
  - name: kvm-host-alerts
    rules:
      - alert: HostHighCPU
        expr: 100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 85
        for: 10m
        labels: { severity: warning }
        annotations: { summary: "Host {{ $labels.instance }} CPU > 85% for 10min" }

      - alert: HostMemoryLow
        expr: (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) < 0.10
        for: 5m
        labels: { severity: critical }
        annotations: { summary: "Host {{ $labels.instance }} memory < 10% available" }

      - alert: HostDiskSpaceLow
        expr: (node_filesystem_avail_bytes{mountpoint="/var/lib/libvirt"} / node_filesystem_size_bytes) < 0.15
        for: 5m
        labels: { severity: critical }
        annotations: { summary: "Libvirt storage < 15% free on {{ $labels.instance }}" }

      - alert: HostDown
        expr: up{job="kvm-hosts"} == 0
        for: 2m
        labels: { severity: critical }
        annotations: { summary: "KVM host {{ $labels.instance }} is unreachable" }

  - name: kvm-vm-alerts
    rules:
      - alert: VMDown
        expr: libvirt_domain_info_state != 1
        for: 5m
        labels: { severity: warning }
        annotations: { summary: "VM {{ $labels.domain }} is not running" }

      - alert: VMHighCPU
        expr: rate(libvirt_domain_info_cpu_time_seconds[5m]) / on(domain) libvirt_domain_info_vcpus > 0.90
        for: 15m
```

```
      labels: { severity: warning }
      annotations: { summary: "VM {{ $labels.domain }} CPU > 90% for 15min" }
```

Alertmanager Routing

alertmanager.yml

```
route:
  receiver: 'default'
  group_by: [alertname, instance]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - match: { severity: critical }
      receiver: 'pagerduty-critical'
    - match: { severity: warning }
      receiver: 'slack-warnings'

receivers:
  - name: 'slack-warnings'
    slack_configs:
      - api_url: 'https://hooks.slack.com/...'
        channel: '#kvm-alerts'
        title: '{{ .GroupLabels.alertname }}'
        text: '{{ .CommonAnnotations.summary }}'

  - name: 'pagerduty-critical'
    pagerduty_configs:
      - service_key: 'xxx-xxx-xxx'

  - name: 'default'
    email_configs:
      - to: 'ops@company.com'
```

Alert Priority Matrix

Alert	Severity	Channel
Host down	**Critical**	PagerDuty + Slack
Memory < 10%	**Critical**	PagerDuty + Slack
Disk < 15%	**Critical**	PagerDuty + Slack
VM not running	Warning	Slack only
CPU > 85%	Warning	Slack only
VM high CPU	Warning	Slack only
Disk forecast < 30d	Info	Email weekly

Log Aggregation with Loki

Centralized logging for KVM hosts and VMs — replaces vRealize Log Insight.

Loki + Promtail Setup

```
# promtail config on each KVM host — /etc/promtail/config.yml
server:
  http_listen_port: 9080

positions:
  filename: /var/lib/promtail/positions.yaml

clients:
  - url: http://loki-server:3100/loki/api/v1/push

scrape_configs:
  - job_name: syslog
    static_configs:
      - targets: [localhost]
        labels:
          job: syslog
          host: kvm-host-01
          __path__: /var/log/messages

  - job_name: libvirt
    static_configs:
      - targets: [localhost]
        labels:
          job: libvirt
          host: kvm-host-01
          __path__: /var/log/libvirt/qemu/*.log

  - job_name: h2kvmctl
    static_configs:
      - targets: [localhost]
        labels:
          job: h2kvmctl
          host: kvm-host-01
          __path__: /var/log/hyper2kvm/*.log
```

LogQL Query Examples

```
# VM start/stop events
{job="libvirt"} |= "starting"
```

Log Retention & Cost

- **vRealize Log Insight:** $4,870/CPU/yr ($50K+ total)
- **Loki:** $0 (open-source, local storage)

```
# Migration errors
{job="h2kvmctl"} |= "error"
  | json | status="failed"

# OOM events on host
{job="syslog"} |= "Out of memory"

# All logs for specific VM
{job="libvirt", host="kvm-host-01"}
  |= "db-server"

# Error rate over time
sum(rate({job="h2kvmctl"}
  |= "error" [5m])) by (host)
```

- Retention: 30 days hot, 1 year cold (S3/MinIO)
- Storage: ~1-5 GB/day for 5 KVM hosts
- Compressed: Loki chunks are 10-20x smaller than raw
- Grafana: unified view of metrics + logs in one UI

Migration from vCenter Monitoring

Map your existing vCenter monitoring to Prometheus/Grafana equivalents.

vCenter Metric → PromQL Mapping

vCenter Metric	PromQL Equivalent	Grafana Panel Type
cpu.usage.average (%)	`100 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100`	Time series, gauge
mem.active.average (KB)	`node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes`	Time series, stat
disk.usage.average (KBps)	`rate(node_disk_read_bytes_total[5m]) + rate(node_disk_written_bytes_total[5m])`	Time series
net.usage.average (KBps)	`rate(node_network_receive_bytes_total[5m]) + rate(node_network_transmit_bytes_total[5m])`	Time series
VM CPU Ready (%)	`rate(libvirt_domain_info_cpu_time_seconds[5m]) / libvirt_domain_info_vcpus`	Time series
Datastore Free Space	`node_filesystem_avail_bytes{mountpoint="/var/lib/libvirt"}`	Gauge, bar chart
VM Power State	`libvirt_domain_info_state`	Status table
Host Uptime	`node_boot_time_seconds`	Stat panel

Open-Source Monitoring: Better Dashboards, Better Alerting, $0 License

Prometheus + Grafana + Loki replaces vCenter monitoring, vROps, and vRealize Log Insight — saving $120K+/yr in licensing. PromQL is more powerful than vCenter's query language. Grafana dashboards are unlimited and shareable. Alertmanager routes to Slack, PagerDuty, email, and webhooks. The open-source monitoring ecosystem has 3,000+ community dashboards and integrates with every tool in your stack.